

Capítulo 6: Sistema de Login com Banco de Dados MySQL

Passo 1: Instalando os "Motores" do Banco de Dados

Primeiro, precisamos instalar os pacotes que ensinam o nosso projeto a conversar com o MySQL.

No terminal do VS Code (com o servidor parado usando Ctrl+C), digite estes dois comandos, um por vez:

```
dotnet add package  
Microsoft.EntityFrameworkCore.Design  
dotnet add package  
Pomelo.EntityFrameworkCore.MySql
```

O pacote Pomelo é o conector oficial e mais rápido para MySQL no .NET.

Passo 2: O Model (A Estrutura do Usuário)

Vamos criar a classe que vai virar a nossa tabela no banco de dados.

1. Na pasta Models, crie um arquivo chamado Usuario.cs:

```
using System.ComponentModel.DataAnnotations;  
  
namespace MeuSite.Models;  
  
public class Usuario  
{
```

```

[Key] // Diz ao banco que este é o ID principal
public int Id { get; set; }

[Required] // Campo obrigatório
public string Nome { get; set; }

[Required]
public string Email { get; set; }

[Required]
public string Senha { get; set; }
}

```

(Nota para a turma: No mercado real, NUNCA salvamos a senha assim, em texto limpo. Usamos criptografia/hash. Mas para o nosso primeiro login na escola, faremos assim para entender a lógica!).

Passo 3: O DbContext e a Conexão

Precisamos de um arquivo que atue como uma "ponte" entre os nossos Models e o MySQL.

1. Crie uma nova pasta chamada Data na raiz do projeto.
2. Dentro dela, crie o arquivo ApplicationDbContext.cs:

```

using Microsoft.EntityFrameworkCore;
using MeuSite.Models;

namespace MeuSite.Data;

public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) :
    base(options) { }

    // Esta linha diz: "Crie uma tabela chamada Usuarios baseada no model
    Usuario"
    public DbSet<Usuario> Usuarios { get; set; }
}

```

3. Agora, vamos dizer onde o banco de dados está. Abra o arquivo appsettings.json (na raiz do projeto) e adicione a "Connection String" (texto de conexão):

```

{
  "ConnectionStrings": {

```

```

    "ConexaoMySQL":
    "Server=localhost;Database=banco_escola;User=root;Password=sua_senha_do_mysql;
    "
    },
    "Logging": { ... }
}

```

(Lembre-se de trocar sua_senha_do_mysql pela senha do banco de vocês na escola).

4. Abra o arquivo Program.cs e avise o sistema que o banco de dados e as Sessões (para manter o usuário logado) existem. Adicione estas linhas antes de `var app = builder.Build();`:

```

using MeuSite.Data;
using Microsoft.EntityFrameworkCore;

// Configura o MySQL
var connectionString =
builder.Configuration.GetConnectionString("ConexaoMySQL");
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseMySQL(connectionString,
ServerVersion.AutoDetect(connectionString)));

// Configura Sessões (para o Login)
builder.Services.AddSession();

```

E adicione `app.UseSession();` antes do `app.UseRouting();`.

Passo 4: O Controller (A Lógica de Login e Cadastro)

Vamos criar o maestro que recebe os dados da tela e manda para o banco.

1. Na pasta Controllers, crie LoginController.cs:

```
using Microsoft.AspNetCore.Mvc;
using MeuSite.Models;
using MeuSite.Data;

namespace MeuSite.Controllers;

public class LoginController : Controller
{
    private readonly AppDbContext _context;

    // Construtor que recebe a conexão com o banco
    public LoginController(AppDbContext context)
    {
        _context = context;
    }

    // 1. Tela de Login (GET)
    public IActionResult Index()
    {
        return View();
    }

    // 2. Recebe os dados de Login (POST)
    [HttpPost]
    public IActionResult Entrar(string email, string senha)
    {
        // Busca no banco um usuário com este email e senha
        var usuario = _context.Usuarios.FirstOrDefault(u => u.Email == email
        && u.Senha == senha);

        if (usuario != null)
        {
            // Login com sucesso! Guarda o nome na sessão e manda pra página
            inicial

            HttpContext.Session.SetString("UsuarioLogado", usuario.Nome);
            return RedirectToAction("Index", "Home");
        }
    }
}
```

```
        ViewBag.Erro = "E-mail ou senha inválidos!";  
        return View("Index");  
    }  
  
    // 3. Tela de Cadastro (GET)  
    public IActionResult Cadastro()  
    {  
        return View();  
    }  
  
    // 4. Salva o novo usuário no banco (POST)  
    [HttpPost]  
    public IActionResult Cadastrar(Usuario novoUsuario)  
    {  
        _context.Usuarios.Add(novoUsuario); // Adiciona na memória  
        _context.SaveChanges(); // Salva no MySQL de verdade  
  
        return RedirectToAction("Index"); // Volta para a tela de login  
    }  
}
```

Passo 5: As Views (Telas do Usuário)

1. Crie a pasta Views/Login.
2. Lá dentro, crie Cadastro.cshtml:

```
<h2>Cadastrar Novo Usuário</h2>
<!-- O "method='post'" esconde os dados ao enviar, ao invés de mostrá-los na URL -->
<form asp-controller="Login" asp-action="Cadastrar" method="post">
  <div class="mb-3">
    <label>Nome</label>
    <input type="text" name="Nome" class="form-control" required />
  </div>
  <div class="mb-3">
    <label>E-mail</label>
    <input type="email" name="Email" class="form-control" required />
  </div>
  <div class="mb-3">
    <label>Senha</label>
    <input type="password" name="Senha" class="form-control" required />
  </div>
  <button type="submit" class="btn btn-success">Salvar Cadastro</button>
</form>
```

3. Crie a tela de login Index.cshtml na mesma pasta Views/Login:

```
<h2>Acesso ao Sistema</h2>

@if(ViewBag.Erro != null)
{
  <div class="alert alert-danger">@ViewBag.Erro</div>
}

<form asp-controller="Login" asp-action="Entrar" method="post">
  <div class="mb-3">
    <label>E-mail</label>
    <input type="email" name="email" class="form-control" required />
  </div>
  <div class="mb-3">
    <label>Senha</label>
    <input type="password" name="senha" class="form-control" required />
  </div>
  <button type="submit" class="btn btn-primary">Entrar</button>
  <a asp-action="Cadastro" class="btn btn-link">Criar conta</a>
</form>
```

Toda a engrenagem (Model, View, Controller e Banco) já está escrita! No entanto, o banco de dados e a tabela *Usuarios* ainda não existem fisicamente no MySQL do computador de vocês.

Para que o C# crie as tabelas automaticamente no MySQL baseadas no nosso código, precisamos rodar os comandos de "Migrations" (Migrações) no terminal. Gostaria que eu listasse os comandos de Migrations necessários para gerar o banco agora mesmo?